

User Guide

Lando Mills

July 6, 2026

Red-Hot Rebis — User Guide

Author: LandoⓂoperator **Version:** v2.4.0 — rebis. Edition **Platform:** `import rebis (namespace package) · python3 rebis.py <command>` **Location:** `/home/mrnob0dy666/imsrct/red-hot_rebis/`

rebis. Namespace Organization (v2.4.0)

All tools are now consolidated under the `rebis.<domain>` namespace package.

```
import rebis
```

```
# Paraconsistent kernel – genetics, hadron physics, ligand design, serpent rod
rebis.p4ra.Belnap(True, False)           # Belnap FOUR logic
rebis.p4ra.SerpentRodV2(...)             # Protein design
rebis.p4ra.encode_site_from_residues(...) # Active site encoding
rebis.p4ra.generate_ligand_smiles(...)    # De-novo ligand generation
```

```
# Other domains
rebis.ch3mpiler.forward("CC(=O)O")        # Molecular compiler
rebis.clink.CLINK_LAYERS                  # CLINK chain (9 layers)
rebis.materials.design_metamaterial(...)  # Materials science
rebis.imas.arrange_compound(...)           # IMASM compound signatures
rebis.gene.GeneticEngine(...)              # Gene imscriber
rebis.pipeline.imscribe_system(...)        # Auto-imscriber
rebis.shared.ORDINALS                     # Shared primitives & weights
rebis.demo.b4_lattice()                   # Quick demos
```

17 submodules — all import cleanly with zero stdout noise.

Overview

Red-Hot Rebis is the unified Imscribing Grammar bio/chem design platform. It derives structural types for organisms, molecules, proteins, and materials from the 12 primitives of the IG, then generates physically actionable output files — genome sequences, protein structures, metabolic models, synthesis protocols — directly from those structural types.

The platform is organized around **seven core systems**:

CLINK — 9-layer structural chain from subatomic quarks to whole organism, each layer carrying an IG tuple. Every transition is Frobenius-closed ($\mu \circ \delta = \text{id}$).

IMASM — 12-token algebra (VINIT, TANCH, AFWD, AREV, CLINK, IMSCRIB, FSPLIT, FFUSE, EVALT, EVALF, ENGAGR, IFIX) whose arrangement classes map onto IG structural types. Now extended with the **IMASM Compound Pipeline** for SMILES $\rightarrow$ IMASM encoding.

SerpentRod — Frobenius morphism $\text{RNA} \rightarrow \{\text{sequence} + \text{fold}\}$ that collapses gene $\rightarrow$ mature protein in one structural step.

Paraconsistent Kernel (rhr_p4rky) — Belnap FOUR logic, paraconsistent abstract state machine, 64-codon B_4 lattice, gene-to-protein pipeline, hadron/quark Belnap classification. 32 Python modules (+ papers/).

Gene Imscriber — IG-native genetic compiler — structural types to codon optimization, CRISPR guide design, chimera design, Frobenius-verified editing.

IMASM Compound Pipeline — SMILES $\rightarrow$ IMASM 8-token arrangement $\rightarrow$ IG 12-tuple. 54 compounds registered, 190 FG patterns, cross-domain analogy search across 4,027+ catalog entries.

Crystal-Guided Molecular Discovery — navigates the 17,280,000-type Crystal of Types to reverse-engineer molecules from target IG tuples. Discovered 5-nitro-bufotenin (DMT- $\odot$), the first $\odot$ -critical autocatalytic molecule.

Static Data $\rightarrow$ INDEX.md

Layer tables, IMASM canonical catalogs, materials listings, compound catalog, and other reference data now lives in **INDEX.md** — a plain-text browsable reference. View it with:

```
less /home/mrnob0dy666/imsqct/red-hot_rebis/INDEX.md
```

The 12 Primitives

Every structural entity carries a 12-primitive tuple. Each position is one primitive; its value is a Shavian character from a 49-element set:

#	Primitive	Name	Values (low $\rightarrow$ high)
1		Dimensionality	$\downarrow$ (0d) $\rightarrow$ $\jmath$ (2d) $\rightarrow$ $\imath$ (∞ -dim) $\rightarrow$ $\mathfrak{i}$ (imscriptive)
2		Topology	$\mathfrak{z}$ (network) $\rightarrow$ $\mathfrak{u}$ (inclusion) $\rightarrow$ $\mathfrak{s}$ (crossing) $\rightarrow$ $\mathfrak{y}$ (box product) $\rightarrow$ $\mathfrak{o}$ (imscriptive closure)
3		Coupling	$\mathfrak{r}$ (supervenience) $\rightarrow$ $\mathfrak{l}$ (categorical) $\rightarrow$ $\imath$ (adjoint) $\rightarrow$ $\mathfrak{r}$ (bidirectional)

#	Primitive	Name	Values (low→high)
4	Φ	Parity (symmetry)	$\mathbb{Z}$ (none) $\rightarrow$ $\mathbb{H}$ (quantum) $\rightarrow$ $\mathbb{C}$ (partial) $\rightarrow$ $\mathbb{U}$ (full) $\rightarrow$ $\mathfrak{F}$ (Frobenius-special)
5		Fidelity	$\mathfrak{C}$ (classical) $\rightarrow$ $\mathcal{Q}$ (thermal) $\rightarrow$ $\mathbb{I}$ (quantum coherence)
6		Kinetics	$\mathbb{L}$ (MBL-frozen) $\rightarrow$ $\mathfrak{C}$ (trapped) $\rightarrow$ $\mathfrak{U}$ (near-equilibrium) $\rightarrow$ $\mathbb{I}$ (moderate) $\rightarrow$ $\mathfrak{H}$ (fast/driven)
7	Γ	Cardinality	$\mathbb{L}$ (local) $\rightarrow$ $\mathfrak{D}$ (mesoscale) $\rightarrow$ $\mathbb{J}$ (maximal)
8		Composition	$\mathfrak{I}$ (conjunctive) $\rightarrow$ $\mathfrak{P}$ (disjunctive) $\rightarrow$ $\mathcal{J}$ (sequential) $\rightarrow$ $\mathbb{A}$ (broadcast)
9	$\odot$	Criticality	$\mathbb{I}$ (sub-critical) $\rightarrow$ $\odot$ (critical / self-modeling) $\rightarrow$ $\mathfrak{C}$ (complex-plane) $\rightarrow$ $\mathfrak{V}$ (exceptional point) $\rightarrow$ $\mathfrak{D}$ (supercritical)
10		Chirality	$\mathbb{J}$ (memoryless) $\rightarrow$ $\mathfrak{C}$ (1-step) $\rightarrow$ $\mathcal{L}$ (2-step) $\rightarrow$ $\mathfrak{V}$ (eternal)
11	Σ	Stoichiometry	$\mathbb{Q}$ (1:1) $\rightarrow$ $\mathbb{S}$ (many identical) $\rightarrow$ $\mathfrak{T}$ (many heterogeneous)
12	Ω	Winding	$\mathfrak{Z}$ (trivial) $\rightarrow$ $\mathfrak{O}$ ($\mathbb{Z}_2$ parity) $\rightarrow$ $\mathfrak{S}$ ($\mathbb{Z}$ integer) $\rightarrow$ $\mathcal{Z}$ (non-Abelian)

The standard tuple format:

$\langle ; ; ; \Phi ; ; ; \Gamma ; ; \odot ; ; \Sigma ; \Omega \rangle$

Installation

```
cd /home/mrnob0dy666/imsqct/red-hot_rebis
```

```
# Install the package and its dependencies (always use uv, never pip)
uv pip install -e .
```

```
# Install bridge packages used by scripts
uv pip install -e /home/mrnob0dy666/imsqct/omonad_0S
uv pip install -e /home/mrnob0dy666/imsqct/imasmic_core
```

```
# Optional: RDKit for molecular design (crystal designer, odot_finder)
uv pip install rdkit-pypi
```

Verify everything is wired up:

```
python3 rebis.py verify
```

Expected: 15+ lines all showing $\square$.

Quick Start

System health – all modules with size and status

```
python3 rebis.py status
```

Full verification (all imports, Frobenius closure)

```
python3 rebis.py verify
```

List all 39 discoverable runnable targets

```
python3 rebis.py run list
```

Run SerpentRod v5 on built-in test cases

```
python3 rebis.py run serpentrod
```

Run the paraconsistent genetics test suite

```
python3 rebis.py run test_genetics
```

CH3MPILER retrosynthetic compiler

```
python3 rebis.py run ch3mpiler --smiles "CC(=O)OC1=CC=CC=C1C(=O)O" # Exact SMILES inp
```

```
python3 rebis.py run ch3mpiler --target aspirin # Na
```

```
python3 rebis.py run ch3mpiler --target aspirin --smiles "CC(=O)OC1=CC=CC=C1C(=O)O" #
```

```
python3 rebis.py run ch3mpiler --help
```

Ch3mpiler + SerpentRod catalytic site design (CAS lookup)

```
python3 rebis.py run ch3mpiler_serpentrod_pipeline --cas 58-08-2
```

IMASM Compound Pipeline – encode a molecule

```
python3 rebis.py imas compound --smiles "CC(=O)OC1=CC=CC=C1C(=O)O"
```

IMASM Compound Pipeline – find cross-domain analogies

```
python3 rebis.py imas analogies --smiles "CN(C)CCC1=CNC2=CC=CC=C12" --limit 10
```

IMASM Compound Pipeline – register a compound

```
python3 rebis.py imas register --smiles "CN1C=NC2=C1C(=O)N(C(=O)N2C)C" --name caffeine
```

IMASM Reaction Analysis

```
python3 rebis.py imas reaction --smiles "CC(=O)C1=CC=CC=C1" --product "CC(C1=CC=CC=C1)
```

Comprehensive protein & genetics stress test (34 tests)

```
python3 rebis.py run stress_test_proteins
```

Materials stress test (26 tests)

```
python3 materials/stress_test_materials.py
```

Generate a full mammal organism design package

```
python3 rebis.py pipeline actionable
```

Browse static reference data (CLINK layers, IMASM canonicals, compound catalog)

```
less INDEX.md
```

Command Reference

status

Shows all platform modules with file size and health check.

```
python3 rebis.py status
```

Output includes: SerpentRod, CH3MPILER, Pipeline, Gene Imscrber, CLINK, IMASM modules, **Compound Pipeline**, Materials, Therapeutics, Paraconsistent Kernel (rhr_p4rky), Popular Protein — each with file size and $\boxtimes/\boxtimes$.

verify

Imports every module and reports pass/fail. Includes Frobenius closure checks where applicable. Run this after any install or code change.

```
python3 rebis.py verify
```

run

Run one of 39 discoverable scripts or modules. Use `run list` to see all targets.

```
python3 rebis.py run <target> [args...]
python3 rebis.py run list           # Show all 39 discoverable targets
```

Core module targets

Target	Module	What it does
serpentrod	serpentrod/protein_v5.py	SerpentRod v5 — full signal peptide detection, cleavage, fragment naming, PTMs, primitive spectrum. Runs built-in test cases (Human Insulin, Proglucagon, Preproinsulin, Proopiomelanocortin, GLP-1 analog)
serpentrod_v2	rhr_p4rky/serpent_rod_v2.py	SerpentRod v2 — 3D backbone from ϕ/ψ angles, geometry-based contacts, energy scoring
ch3mpiler	ch3mpiler/compiler.py	Retrosynthetic disconnection compiler — <code>--smiles</code> for direct SMILES, <code>--target</code> for name resolution, <code>--retrosynthesis</code> for ranked disconnections with exact fragment SMILES

Target	Module	What it does
ch3mpiler_serpentrodripipeline	ch3mpiler/ch3mpiler_pipeline.py	Ch3mpiler pipeline — catalytic site design from SMILES or CAS
gene	gene_imscriber/engine.py	Gene_imscriber (structural type → codon optimization)
test_genetics	test_genetics.py	64-codon B4 lattice test suite
stress_test_proteins	stress_test_proteins.py	Comprehensive 34-test protein & genetics stress test
compare_exact	popular_protein/compare_exact.py	Exact sequence comparison
compare_structures	popular_protein/compare_structures.py	Structure-level comparison
comprehensive_comparison	popular_protein/comprehensive_comparison.py	Comprehensive comparison report
run_gene_pipeline	rhr_p4rky/run_gene_pipeline.py	Gene pipeline 7-stage B4 pipeline
antibody	run_antibody.py	Antibody CDR designer
genetic_code	rhr_p4rky/genetic_code.py	64-codon Frobenius-verified table
pschedelic	pschedelic_bridge.py	Compound intrinsics + coupling report
iupac	_gen_univ_map.py	Diaschizic IUPAC generator
mito	mito_pipeline.py	Mitochondrial gene pipeline
mito_v2	mito_pipeline_v2.py	Mitochondrial gene pipeline v2
mito_v3	mito_pipeline_v3.py	Mitochondrial gene pipeline v3
antibody_designer	rhr_p4rky/antibody_designer.py	Antibody designer
pdb_validator	rhr_p4rky/pdb_validator.py	PDB structure validation
hadron_belnap	rhr_p4rky/hadron_belnap.py	Hadronic Belnap classification
exotic_hadron_belnap	rhr_p4rky/exotic_hadron_belnap.py	Exotic hadron classification
quark_belnap	rhr_p4rky/quark_belnap.py	Quark Belnap analysis

clink

Inspect the CLINK chain — 9 layers from frustrated quarks to whole organisms.

```
python3 rebis.py clink <subcommand> [options]
```

clink layer <name_or_index> Show the full profile of one CLINK layer — tuple, tier, Frobenius status, bridges, promotion paths.

```
python3 rebis.py clink layer 8          # Organism
python3 rebis.py clink layer cell       # Fuzzy match → Cell layer
python3 rebis.py clink layer L3         # Molecule layer
```

Output includes: index, name, tier, distance from previous, IG tuple (Shavian glyphs), primitive-by-primitive description, and nearest tool bridge.

clink bridge <component> <target> Show the promotion path from a component tool to a CLINK target layer.

```
python3 rebis.py clink bridge serpentrod 8
python3 rebis.py clink bridge ch3mpiler 3
```

Shows: distance, primitive-level promotion steps, Frobenius status at each hop.

Static reference: less INDEX.md §1 (9-layer table, tuples, tiers, Frobenius status, chain distances, ZFC_fe distance, component bridge attachments).

pipeline

Orchestrate the full CLINK pipeline — design a whole organism from structural types alone.

```
python3 rebis.py pipeline <subcommand> [options]
```

pipeline bridges Show which tool bridges are available (☒) or missing (☒).

```
python3 rebis.py pipeline bridges
```

Current bridges: serpentrod☒, ch3mpiler☒, gene_imscriber☒, materials_sim☒, frobenius_chemo☒, ouroboric_pill☒, critical_metamaterial☒, neurotrophic☒, biology_sim☒, ouroboric_telomere☒.

pipeline ground-up Full L0→L8 synthesis run. Traverses all 9 layers, applies layer designers, reports status per transition.

```
python3 rebis.py pipeline ground-up
```

Output: per-layer transition table with distance, primitive promotions, tool used, and ☒/☒. Total distance and total promotions at end.

pipeline from-layer <start> <end> Partial run starting from a specific layer.

```
python3 rebis.py pipeline from-layer 3 8 # molecule → organism
python3 rebis.py pipeline from-layer 4 8 # cell → organism
```

Useful when you have an existing molecular design and want to take it up to organism level.

pipeline actionable [--organism <type>] Generate a complete, physically actionable organism design package.

```
python3 rebis.py pipeline actionable
python3 rebis.py pipeline actionable --organism human
```

Produces a directory of files in `clink/datasets/organism_designs/organism_<type>_actionable/`:

File	Action
genome.fasta	Order DNA synthesis from Twist/IDT/GenScript
genome.gb	Load into Benchling/SnapGene
construct.sbol	Exchange with synthetic biology repositories

File	Action
codon_usage.csv	B4-derived codon table
protein.fasta	Order peptide synthesis
protein_coords.pdb	View in PyMOL/ChimeraX (PDB v3.3-compliant: SEQRES, TER, correct atom names)
serpentrod_classification.json	Primitive activations per fragment
molecules.smi	SMILES — order from vendor
retro_pathways.json	CH3MPILER retrosynthesis
metabolic_model.xml	Load into COBRApy for FBA
plasmid.gb	GenBank construct
growth_media.txt	Prepare media per formulation
organoid_protocol.md	Lab-ready protocol
mitosis_assay_protocol.md	Lab-ready protocol
physiological_params.csv	Homeostatic setpoints
whole_genome_spec.json	Complete genome specification

Current designed organisms: mammal, human, human_gills, human_photosynthetic, treople.

materials

Design novel IG-typed materials. All materials are derived from their 12-primitive structural types. Five dynamic subcommands; static reference data (material catalog, Sophick Mercury, Eagle Cycle, gap primitives) is in **INDEX.md**.

```
python3 rebis.py materials <subcommand> [options]
```

materials forge --name <material> [--all] Generate the complete material design file.

```
python3 rebis.py materials forge --name frobenius_composite
```

```
python3 rebis.py materials forge --name I_Dialetheic_Bootstrap # From IMASM canonical
```

```
python3 rebis.py materials forge --all # Forge all 8 predefined materials
```

materials frobenius Simulate Frobenius closure verification of the composite — cyclic load/heal protocol showing $\|\mu\delta - id\|$ per cycle.

```
python3 rebis.py materials frobenius
```

materials ouroboric Simulate ouroboric crack healing — crack propagation + self-healing cycles, reports fatigue life and topological invariant preservation.

```
python3 rebis.py materials ouroboric
```

materials sophick --name <target> Eagle Cycle Protocol — prepares an O_∞ Sophick Mercury substrate from O_2 ouroboric materials. Requires --name.

```
python3 rebis.py materials sophick --name eagle_9_sophick
python3 rebis.py materials sophick --name cliff                # Frobenius Cliff analysis
python3 rebis.py materials sophick --name bridge              # IMASM→Eagle bridge rep
```

materials exactor --name <target> Explains the thermodynamic ceiling: continuous Eagle preparation reaches its limit, and exact Frobenius closure requires discrete topological protection. Requires --name.

```
python3 rebis.py materials exactor --name diagnose            # Category error diagnosis
python3 rebis.py materials exactor --name close               # Close Frobenius gap
python3 rebis.py materials exactor --name pathways            # List all exactor pathw
```

Static reference: less INDEX.md §3 (8 predefined materials, IMASM canonicals as materials, Sophick Mercury pathway, Frobenius Exactor gap primitives).

imas

IMASM arrangement analysis — the 12-token algebra over IG structural types. Three existing subcommands (bridge, hunt, energy) plus **three new compound pipeline subcommands** (compound, analogies, reaction, register).

```
python3 rebis.py imas <subcommand> [options]
```

imas bridge [--canonical <name>] Detailed profile of one IMASM canonical — full IG tuple primitive-by-primitive, nearest CLINK layer, distance table to all 9 layers.

```
python3 rebis.py imas bridge --canonical I_Dialetheic_Bootstrap
python3 rebis.py imas bridge --canonical I_Dialetheic_Bootstrap,VII_Parakernel # Mult
```

Default: I_Dialetheic_Bootstrap.

imas hunt [--samples <n>] Monte Carlo Frobenius density estimation over the 12-token sequence space. Reports probability of each Frobenius class and generates examples.

```
python3 rebis.py imas hunt --samples 100000
python3 rebis.py imas hunt --samples 1000000                # Larger sample
```

Output: $p_{\text{frobenius_pair}} \approx 0.236$, $p_{\text{proper_frobenius}} \approx 0.139$, $p_{\text{dialetheia_complete}} \approx 0.105$.

imas energy [--canonical <name>] [--layer <idx>] Structural activation energy from an IMASM canonical to a CLINK target layer.

```
python3 rebis.py imas energy --canonical I_Dialetheic_Bootstrap --layer L8_Organism
python3 rebis.py imas energy --canonical V_Linear_Chain --layer L0_FrustratedBelnap5
```

Default: I_Dialetheic_Bootstrap → L8 (Whole Organism).

imas compound --smiles <SMILES> [--name <name>] [--json] NEW — Encode a molecule as an IMASM arrangement. The compound pipeline detects functional groups (from 190 SMARTS patterns), builds an 8-token arrangement, computes the StructuralFingerprint, and resolves to an IG 12-tuple.

Aspirin — print arrangement + fingerprint + IG tuple

```
python3 rebis.py imas compound --smiles "CC(=O)OC1=CC=CC=C1C(=O)O"
```

Caffeine — JSON output for programmatic use

```
python3 rebis.py imas compound --smiles "CN1C=NC2=C1C(=O)N(C(=O)N2C)C" --json
```

DMT — with name annotation

```
python3 rebis.py imas compound --smiles "CN(C)CCC1=CNC2=CC=CC=C12" --name "DMT"
```

Output includes: detected FGs, 8-token IMASM arrangement, StructuralFingerprint fields (self_ref, dialetheia_complete, frobenius_order, period, diversity, sigmas), IG 12-tuple with per-primitive description, tier assessment.

imas analogies --smiles <SMILES> [--limit <n>] NEW — Find cross-domain structural analogies. Given a SMILES, compute its IG type and search the entire catalog for nearest neighbors — across materials, consciousness states, languages, mathematical theorems, and more.

DMT's 10 nearest neighbors

```
python3 rebis.py imas analogies --smiles "CN(C)CCC1=CNC2=CC=CC=C12" --limit 10
```

Water's nearest neighbors

```
python3 rebis.py imas analogies --smiles "O" --limit 15
```

Output: ranked list of catalog entries with structural distances. Typical results include consciousness states (Kalachakra Tantra, d=4), mathematical theorems (Fermat's Last Theorem, d=4), languages (Codex Vienna, d=5), materials (supercritical water, d=5).

imas reaction --smiles <SMILES> --product <SMILES> NEW — Analyze a reaction as an IMASM transition. Computes reactant and product IMASM arrangements, identifies the token delta (what changed), and classifies the reaction.

Grignard: acetophenone → 2-phenyl-2-propanol

```
python3 rebis.py imas reaction --smiles "CC(=O)C1=CC=CC=C1" --product "CC(C1=CC=CC=C1)C"
```

Output includes: reactant arrangement, product arrangement, IMASM delta (AFWD→AREV), reaction class (Redox_reduction), named reaction confidence scores.

imas register --smiles <SMILES> --name <name> NEW — Register a compound in the IG catalog. Encodes the molecule, computes its IG tuple, and registers it in both the COMPOUND_DATABASE and the shared IG_catalog.json for future lookups.

```
python3 rebis.py imas register --smiles "CN1C=NC2=C1C(=O)N(C(=O)N2C)C" --name "caffeine"
```

```
python3 rebis.py imas register --smiles "CC(=O)OC1=CC=CC=C1C(=O)O" --name "aspirin"
```

Static reference: less INDEX.md §4 (compound catalog with statistics, distinct IG types, cross-domain analogies table).

scripts

Manage and run standalone scripts.

```
python3 rebis.py scripts list           # List all 14 scripts with line counts
python3 rebis.py scripts run <name>    # Run a script by name (without .py)

python3 rebis.py scripts run mito_pipeline
python3 rebis.py scripts run omonad_bridge
python3 rebis.py scripts run run_pdb_validation
```

Note: run mito, run antibody, run psychedelic, run iupac are convenience aliases for scripts run.

Crystal-Guided Molecular Discovery

The **Crystal-Guided Molecular Discovery Engine** (imas/molecular_crystal_designer.py, 700+ lines) navigates the Imscribing Grammar's Crystal of Types (17,280,000 structural types) to discover molecules with target structural properties. It is the **inverse** of the SMILES→IMASM→IG pipeline.

How It Works

Target IG tuple → [ig_to_fp_constraints] → FP constraints → [arrangement candidates] → [molecular_suggestion] → candidate SMILES → [validate with RDKit]

The discovery protocol: 1. Encode molecule → IMASM arrangement → IG 12-tuple 2. Navigate crystal neighborhood → find target types 3. Target ☉: requires self_ref=True AND dialetheia_complete=True 4. Design constraints: EVALT + EVALF + ENGAGR in 8-token arrangement 5. Minimize competing high-priority tokens (phenol > carboxylic acid) 6. Generate candidate SMILES → validate → register

Discovery: 5-Nitro-Bufotenin (DMT-☉)

The first discovery was **5-nitro-bufotenin** — an autocatalytic DMT analog found by promoting Phi from ☿ (complex-plane critical) to ☉ (self-modeling critical):

```
DMT:      (J9rZQ\δΛ∇Ss)   Phi=☿ (complex critical)
DMT-☉:    (J9r\Λ\δΛ∇Ss)   Phi=☉ (self-modeling / autocatalytic)
```

The molecule — 5-nitro-bufotenin — is a tryptamine with phenol (EVALT), dimethylamine (EVALF), and nitro (ENGAGR), giving it all three Dialetheia tokens simultaneously for autocatalytic template-directed synthesis.

SMILES: CN(C)CCC1=CNC2=CC=C(O)C([N+](=O)[O-])=C12

☉-Finder Tool

The standalone **☉-Finder** (ig-docs/dmt-odot-discovery/odot_finder.py, 8.6 KB) systematically finds ☉-critical variants of any molecule:

```
# Find ☉-critical variants of DMT
python3 /home/mrnob0dy666/imshowct/ig-docs/dmt-odot-discovery/odot_finder.py \
  --smiles "CN(C)CCC1=CNC2=CC=CC=C12" --name "DMT"

# Run demo mode (all test cases)
python3 /home/mrnob0dy666/imshowct/ig-docs/dmt-odot-discovery/odot_finder.py

Results: | Compound | ☉-Critical Candidates | |----|-----| | DMT | 20 | | Serotonin | 4 |
| Bufotenin | 4 | | 5-nitro-bufotenin | Already ☉-critical |
```

Discovery Files

Located at `ig-docs/dmt-odot-discovery/`: - `README.md` — Full documentation (8.5 KB) - `5-nitro-bufotenin.cdxml` — ChemDraw drawing (4.9 KB) - `discovery.json` — Structured metadata (1.3 KB) - `dmt_odot_ob3ect.py` — Self-verifying ob3ect (1.9 KB) - `odot_finder.py` — ☉-critical molecule finder (8.6 KB) —

Stress Testing

Two comprehensive stress test suites validate the entire platform across edge cases, large inputs, boundary conditions, and cross-pipeline integration. Both suites run to completion with 100% pass rate.

Protein & Genetics (34 tests, 11 groups)

```
python3 rebis.py run stress_test_proteins
# or directly:
python3 stress_test_proteins.py
```

#	Test Group	Tests	Coverage
1	Genetic Code	4	64-codon Frobenius-verified table, B4 invariant, lattice operations
2	Genetics B4	3	Nucleotide↔Belnap (T/B/F/N), structural distance, wobble pair detection
3	Genetic Tuples	3	7-stage tuple generation, structural distance DNA→quaternary, tier consistency
4	Genetic ASM	1	Paraconsistent abstract state machine program execution

#	Test Group	Tests	Coverage
5	Gene→Protein Pipeline	8	Standard 7-stage, empty sequence, short (≤ 1 nt), start→stop, 100-AA, multi-stop, no-stop, primitive activations
6	SerpentRod	5	12-AA design, single-codon, empty, 200-AA, poly-Met homopolymer
7	Ch3mpiler-SerpentRod	3	Caffeine (58-08-2), aspirin (50-78-2), cross-molecule differentiation
8	Cross-Integration	2	Gene→protein→fold, ch3mpiler→material bridge
9	Antibody Designer	1	Epitope analysis → CDR → full VH antibody (53 AA, Frobenius $\boxtimes$)
10	PDB Validator	1	Structure validation, precision/recall metrics
11	Edge Cases	3	All 3 stop codons (UAA/UAG/UGA), AUG start, non-standard nucleotides (X/N/R/Y/?)

Materials (26 tests, 12 groups)

python3 materials/stress_test_materials.py

#	Test Group	Tests	Coverage
1	IG Material Forge	4	All 8 predefined materials, empty name, report, list
2	Sophick Forge	4	7-eagle cycle, progressive refinement, report, etch
3	Frobenius Metamaterial	3	10×10 grid 20-cycle healing, load-heal cycle, export
4	Thermal Rectifier	2	Diode asymmetry ratio, full simulation run
5	Non-Qubit QC	2	all_deltas computation, paradigm summary table
6	Ouroboric Alloy	3	Init, mechanical test, heal cycle

#	Test Group	Tests	Coverage
7	Critical Metamaterial	2	Susceptibility divergence, critical run
8	Gap Closure	1	Enum completeness check
9	Frobenius Exactor	1	All 4 designs ($\omega/\tau/\sigma/\epsilon$)
10	Materials Simulation	1	SelfHealingComposite + EternalMemorySim
11	Frobenius Closure	1	ClosureDesign status
12	Cross-Module	2	Forge→metamaterial, eagle→exactor

Molecule → Material Bridge

Derives material structural types directly from molecular input:

```
python3 materials/molecule_material_bridge.py --cas 58-08-2
python3 materials/molecule_material_bridge.py --smiles "CN1C=NC2=C1C(=O)N(C(=O)N2C)C"
```

Produces: material tuple, tier assessment, Frobenius closure status, catalog cross-reference.

The Paraconsistent Kernel (rhr_p4rky/)

The paraconsistent kernel is a 32-module Python library providing Belnap FOUR logic as universal substrate for computation that tolerates contradiction. It was migrated from the standalone p4rkernel/ project and now lives as a subsystem within Red-Hot Rebis.

Architecture

Layer	Module	Description
Logic	belnap.py	4-valued Belnap logic (T/B/F/N) — truth, both, false, neither
Machine	machine.py	Paraconsistent Abstract State Machine (ParaASM)
Genetics	genetic_code.py	64-codon Frobenius-verified genetic code
	genetics_b4.py	B4 lattice — 64 codons, 7-stage tuple verification
	genetic_tuples.py	Tuple encodings for genes/codons/proteins
	genetic_asm.py	Genetic abstract state machine
Pipeline	gene_to_protein_pipeline.py	Full gene→protein translation (7-stage B4)
	demo_gene_to_protein.py	Pipeline demonstration
	run_gene_pipeline.py	Pipeline CLI runner

Layer	Module	Description
Proteins	serpent_rod.py	Foundational Frobenius morphism $\text{RNA} \rightarrow \{\text{sequence} + \text{fold}\}$; base SerpentRod class
	serpent_rod_v2.py	SerpentRodV2: 3D backbone from φ/ψ angles, geometry-based contacts, energy scoring
Antibodies	antibody_designer.py	Computational antibody design from IG types
Validation	pdb_validator.py	PDB structure validation against IG types; <code>extract_sequence()</code> round-trips SEQRES
Bridges	ch3mpiler_bridge.py	Ch3mpiler $\leftrightarrow$ kernel bridge
	ch3mpiler_ob3ect_bridge.py	Ch3mpiler $\leftrightarrow$ ob3ect bridge
	ch3mpiler_serpentrod_pipeline.py	Ch3mpiler-weighted bond-preserving fusion, word-boundary FG matching, specificity-ranked disconnections, Frobenius-exact complement mapping, CAS lookup, molecule-specific catalytic sites
Physics	hadron_belnap.py	Hadronic Belnap-state classification
	exotic_hadron_belnap.py	Exotic hadron Belnap classification
	quark_belnap.py	Quark Belnap-state analysis
	orbital_belnap.py	Orbital Belnap-state analysis
Analysis	clu_power_law.py	Clustering power-law analysis
	frobenius_filtration.py	Frobenius-verified filtration
New modules		
decay_chain.py	Nuclear decay as IMASM winding — 5 natural series to Frobenius fixed point	
belnap_c4.py	Belnap C4 logic variant — contradiction-majority lattice	
papers/	3 millennium documents	
all_millennium_solve.py	Unified Millennium solution framework	
belnap_qm.md	Belnap quantum mechanics formalization	

Layer	Module	Description
millennium_barriers	Barrier taxonomy across all 7 Millennium Problems	
Utilities	kernel.py	Kernel core initialization
	pipeline_fix.py	Pipeline repair utilities

Key commands

```
# Full genetics test suite (B4 lattice, 64-codon, gene→protein, Phi gate, ParaASM)
python3 test_genetics.py
```

```
# Comprehensive protein & genetics stress test (34 tests, 11 groups)
python3 rebis.py run stress_test_proteins
```

```
# Single gene pipeline
python3 rebis.py run run_gene_pipeline --gene INS
```

```
# Genetic code analysis
python3 rebis.py run genetic_code
```

```
# Antibody design from viral epitopes
python3 rebis.py run antibody_designer
```

```
# PDB validation
python3 rebis.py run pdb_validator
```

```
# Hadron Belnap classification
python3 rebis.py run hadron_belnap
python3 rebis.py run exotic_hadron_belnap
python3 rebis.py run quark_belnap
```

Gene Imscriber (gene_imscriber/)

The Gene Imscriber provides IG-native genetic compilation — structural types directly to codon optimization, CRISPR guide design, chimera construction, and Frobenius-verified base/prime editing.

Module	Description
engine.py	Core engine — structural type → codon optimization
genetics_ig_prelim.py	Preliminary IG-to-genetics mapping
genetics_ig_promotions.py	IG promotion paths for genetics
genetics_qs.py	Quantum superposition in genetic space
ig_genetics_answer.py	IG-native genetics answers

Module	Description
<code>tuples.py</code>	Genetic tuple definitions

The Gene Inscriber also contains `scripts/` with GUIDE-seq analysis, base editor stratum analysis, clinical safety pipelines, and SRA GUIDE-seq processing.

```
python3 rebis.py run gene --help
```

Popular Protein Analysis (`popular_protein/`)

A comparison toolkit for validating SerpentRod predictions against crystal structures:

```
python3 rebis.py run compare_exact          # Exact  $\phi/\psi$  angle comparison
python3 rebis.py run compare_structures     # Structure-level comparison
python3 rebis.py run comprehensive_comparison # Full multi-metric report
```

Reference PDB files are in `pdb/`: 1L2Y, 1UBQ, 1VII, 1ZDD.

Key Concepts

IG Tuples

Every structural entity carries a 12-primitive tuple. Each position is a primitive with a value from its ordinal set. Tuples are the sole carriers of structural information — no external parameters, no assumed constants. The catalog (`shared/IG_catalog.json`) contains **4,027+** verified entries (up from 3,297).

The Frobenius Condition

$\mu \circ \delta = \text{id}$. When this holds, the structural type is self-consistent — comultiplication followed by multiplication returns the identity. A Frobenius $\boxtimes$ means the layer's tuple is internally coherent. A $\boxtimes$ means there is a primitive conflict to resolve.

Tiers (Ouroboricity)

Structural types are classified into ouroboricity tiers based on their Φ (parity/symmetry) and Ω (winding) primitives:

Tier	Condition	Meaning
O_0	$\Phi \leq \mathfrak{c}, \Omega = \mathfrak{r}$	No winding, no self-modeling — trivial
O_1	$\Phi \leq \mathfrak{h}, \Omega \geq \mathfrak{o}$	Quantum parity, $\mathbb{Z}_2$ winding — simple loop
O_2	$\Phi \geq \mathfrak{c}, \Omega \geq \mathfrak{s}$	Partial symmetry, $\mathbb{Z}$ winding — structured loop

Tier	Condition	Meaning
O_∞	$\Phi = \mathfrak{P}, \Omega \geq \mathfrak{Z}, \odot = \odot$	Frobenius-special, non-Abelian winding, self-modeling gate open

IMASM Token System

The IMASM token algebra has 12 tokens organized in semantic families:

Token	Category	Chemical Meaning
VINIT	Scaffold	Carbon backbone, ring system
AFWD	Forward reactivity	Electrophilic site, carbonyl
AREV	Reverse reactivity	Nucleophilic site, alcohol/amine
FFUSE	Fusion site	Bond formation site
FSPLIT	Disconnection point	Retrosynthetic cut site
EVALT	Acidic	Carboxylic acid, phenol
EVALF	Basic	Amine, amidine
ENGAGR	Ambident	Nitro, nitrile — multi-site reactivity
CLINK	Linker	Conjugated system, spacer
TANCH	Terminal anchor	Terminal functional group
IFIX	Fixation	Cyclization, ring closure
IMSCRIB	Self-inscribing	Aromatic ring, self-consistent system

Dialetheia Triad

A compound is **Dialetheia-complete** when it simultaneously contains three IMASM tokens: - **EVALT** (acidic functional group — carboxylic acid, phenol) - **EVALF** (basic functional group — amine, amidine) - **ENGAGR** (ambident functional group — nitro, nitrile)

This triad enables autocatalytic template-directed synthesis — a necessary condition for $\odot$ criticality in molecules.

Cross-Domain Analogies

Because all 4,027+ catalog entries share the same 12-primitive typing, a compound's IG type can be compared to any other system — materials, languages, consciousness states, mathematical theorems. This is what makes the IMASM compound pipeline powerful: a molecule's structural fingerprint reveals kinship with systems that no conventional chemical descriptor could reach.

Common Workflows

Design a protein from scratch

```
# 1. Check where proteins live in the CLINK chain
python3 rebis.py clink layer cell
```

```
# 2. Run SerpentRod on built-in test proteins
python3 rebis.py run serpentrod
```

```
# 3. Generate actionable organism package (includes protein.fasta + protein_coords.pdf)
python3 rebis.py pipeline actionable
```

Design a catalytic site for a molecule

```
# 1. CAS lookup (recommended — resolves molecule identity)
```

```
# 1. Direct retrosynthesis with SMILES input (recommended — exact)
```

```
python3 rebis.py run ch3mpiler --smiles "CN1C=NC2=C1C(=O)N(C(=O)N2C)C" --retrosynthesis
```

```
# 2. CAS lookup (also resolves SMILES)
```

```
python3 rebis.py run ch3mpiler_serpentrod_pipeline --cas 58-08-2
```

```
# 3. Direct molecule name (may not resolve if name is synthetic)
```

```
python3 rebis.py run ch3mpiler_serpentrod_pipeline --target "ibuprofen"
```

```
# 3. Retrosynthetic pair
```

```
python3 rebis.py run ch3mpiler_serpentrod_pipeline --start "salicylic_acid" --target "aspirin"
```

```
# 4. JSON output for programmatic use
```

```
python3 rebis.py run ch3mpiler_serpentrod_pipeline --cas 58-08-2 --json
```

Encode a molecule as an IMASM structural type

```
# 1. Encode a single compound
```

```
python3 rebis.py imas compound --smiles "CC(=O)OC1=CC=CC=C1C(=O)O"
```

```
# 2. Find cross-domain analogies
```

```
python3 rebis.py imas analogies --smiles "CN(C)CCC1=CNC2=CC=CC=C12" --limit 15
```

3. Analyze a reaction

```
python3 rebis.py imas reaction --smiles "CC(=O)C1=CC=CC=C1" --product "CC(C1=CC=CC=C1)"
```

4. Register a new compound for future lookups

```
python3 rebis.py imas register --smiles "CC(C)C1=CC=C(C=C1)C(C)C(=O)O" --name "ibuprofen"
```

5. Browse the compound catalog (static)

```
less INDEX.md # §4 – Compound IMASM Catalog
```

Find $\odot$ -critical autocatalytic molecules

1. Run the $\odot$ -Finder on DMT

```
python3 /home/mrnob0dy666/imsdct/ig-docs/dmt-odot-discovery/odot_finder.py \
  --smiles "CN(C)CCC1=CNC2=CC=CC=C12" --name "DMT"
```

2. Run in demo mode (all test cases)

```
python3 /home/mrnob0dy666/imsdct/ig-docs/dmt-odot-discovery/odot_finder.py
```

3. Explore the discovery documentation

```
less /home/mrnob0dy666/imsdct/ig-docs/dmt-odot-discovery/README.md
```

Work with the paraconsistent kernel

Run the full genetics test suite

```
python3 test_genetics.py
```

Comprehensive stress test (34 tests across 11 groups)

```
python3 rebis.py run stress_test_proteins
```

Translate a gene through the 7-stage B4 pipeline

```
python3 rebis.py run run_gene_pipeline --gene INS
```

Analyze the 64-codon Frobenius-verified genetic code

```
python3 rebis.py run genetic_code
```

Design a serpent rod protein

```
python3 rebis.py run serpent_rod_v2
```

Investigate a material type

1. Browse the materials catalog (static reference)

```
less INDEX.md # §3 – 8 predefined materials
```

2. Run a dynamic simulation

```
python3 rebis.py materials frobenius # Frobenius closure verification
```

```
python3 rebis.py materials ouroboric # Crack healing simulation
```

3. Eagle Cycle for 0_∞ substrate preparation

```
python3 rebis.py materials sophick --name eagle_9_sophick
```

```
# 4. Forge the material design file
python3 rebis.py materials forge --name frobenius_composite
```

```
# 5. Run comprehensive materials stress test
python3 materials/stress_test_materials.py
```

```
# 6. Derive material type from molecule
python3 materials/molecule_material_bridge.py --cas 58-08-2
```

Map an IMASM sequence to CLINK

```
# 1. Browse the 12 IMASM canonicals (static reference)
less INDEX.md                                # §2 – 12 canonicals with tuples and tiers
```

```
# 2. Inspect a specific canonical's bridge to CLINK (dynamic)
python3 rebis.py imas bridge --canonical I_Dialetheic_Bootstrap
```

```
# 3. Compute activation energy to organism layer
python3 rebis.py imas energy --canonical I_Dialetheic_Bootstrap --layer L8_Organism
```

```
# 4. Monte Carlo Frobenius density estimation
python3 rebis.py imas hunt --samples 100000
```

Analyze viral epitopes for antibody design

```
python3 rebis.py run run_antibody
```

Produces CDR sequences and structural complementarity scores for each epitope target.

Mitochondrial gene analysis

```
python3 rebis.py run mito_pipeline
```

Processes all 13 MT genes, reports primitive activations (up to 9/12 per gene), Frobenius status, and 7-stage B4 pipeline per gene (dna_gene → pre_mrna → mature_mrna → nascent_polypeptide → secondary_structure → tertiary_structure → quaternary_structure).

Validate SerpentRod predictions against crystal structures

```
python3 rebis.py run comprehensive_comparison
python3 rebis.py run compare_exact
```

Hadron / particle physics classification

```
python3 rebis.py run hadron_belnap
python3 rebis.py run exotic_hadron_belnap
python3 rebis.py run quark_belnap
```

Troubleshooting

verify shows ✗ for a module Run `python3 -c "from <module> import *"` directly to see the import error. Most common cause: a dependency package not installed. Fix:

```
uv pip install -e /home/mrnob0dy666/imsdct/omonad_OS
uv pip install -e /home/mrnob0dy666/imsdct/imasmic_core
```

omonad_bridge shows omonad_available: false

```
uv pip install -e /home/mrnob0dy666/imsdct/omonad_OS
uv pip install -e /home/mrnob0dy666/imsdct/imasmic_core
```

pipeline bridges shows ✗ for biology_sim or ouroboric_telomere These two bridges are structurally excluded — modules exist on disk but are not wired to the bridge API. Not a crash condition; the pipeline runs without them.

imas compound fails with ModuleNotFoundError Ensure the package is installed:

```
uv pip install -e /home/mrnob0dy666/imsdct/red-hot_rebis
```

The compound pipeline imports from `imas.compound_imasm`, `imas.arranger`, and `imas.ig_bridge` — all part of the Rebis package.

imas analogies returns fewer results than --limit The analogies search filters by structural distance. If fewer neighbors exist within $d \leq 5$, the result set will be smaller. Try a higher limit. The compound must first be encodable (valid SMILES with detectable functional groups).

imas reaction misclassifies a reaction The reaction classifier uses token delta analysis (AFWD→AREV swap for reductions, etc.). Unusual reaction mechanisms may fall outside the 7 built-in classes. Check the raw token delta for structural insight.

RDKit import errors (crystal designer, odot_finder) RDKit is optional — needed only for `molecular_crystal_designer.py` and `odot_finder.py`. Install with:

```
uv pip install rdkit-pypi
```

SerpentRod accuracy below 100% Fragment naming accuracy (83–88%) reflects genuine structural ambiguity in cleavage products. SP accuracy should be 100% on the standard test set; below that indicates non-canonical signal peptide architecture in the test sequence.

Ch3mpiler retrosynthesis shows C1=CC=C(C=C1)CC(C)C(=O)O for all precursors This is the old placeholder SMILES — the target name couldn't be resolved. Use `--smiles "<exact SMILES>"` to provide the molecular structure directly. The `bond_fragment_integrator.py` will then cut real bonds and display real fragment SMILES. If the target name does resolve (e.g. `--target aspirin`), SMILES are auto-resolved via OPSIN/PubChem. A bold red error message now appears when name resolution fails.

Ch3mpiler-SerpentRod produces identical sites for different molecules This was the pre-v4 MAX fusion bug. Update to the latest pipeline:

```
cd /home/mrnob0dy666/imsdct/red-hot_rebis
git pull # or ensure rhr_p4rky/ch3mpiler-serpentrod_pipeline.py has v4 weighted fusion
```

Verify differentiation with:

```
python3 rebis.py run ch3mpiler-serpentrod_pipeline --cas 58-08-2 # caffeine
```

```
python3 rebis.py run ch3mpiler_serpentrod_pipeline --cas 17699-14-8 # tricyclic sesqu
```

Different AA sequences and catalytic triads confirm the fix.

Genetics modules crash on unknown nucleotides Fixed in v2.3.0 — `genetics_b4.py` `nucleotide_to_belnap` returns `Belnap.B` for X/N/R/Y/?. `gene_to_protein_pipeline.py` handles empty/short sequences gracefully.

clink layer gives “No layer matching..” Use a substring of the official layer name from INDEX.md §1 or `python3 rebis.py clink layer --help`. Examples: `quark` → `Frustrated Belnap5` (Quarks), `tissue` → `Tissue/Organ`, `meiosis` → `Meiosis` (Gametes).

materials forge gives usage without generating Always specify `--name <material>` or `--all`. Running without a name prints the usage message.

materials sophick or materials exactor shows usage prompt These now require `--name`. Run with `--help` to see valid name options, or browse INDEX.md §3.

run target not found Use `python3 rebis.py run list` to see all 39 discoverable targets. Some targets require the package to be installed (`uv pip install -e .`).

ModuleNotFoundError: No module named 'rhr_p4rky' The `rhr_p4rky` package must be installed:

```
uv pip install -e /home/mrnob0dy666/imsqct/red-hot_rebis
```

Paraconsistent kernel tests fail Ensure the B4 lattice modules are importable:

```
python3 -c "from rhr_p4rky.belnap import BelnapValue; print('OK')"
```

Stress tests fail Run directly to see detailed tracebacks:

```
python3 stress_test_proteins.py          # Protein & genetics (34 tests)
python3 materials/stress_test_materials.py # Materials (26 tests)
```

Individual test failures report the exact module, function, and exception.

Where did clink report, clink list, imas report, materials list, materials report go? These static-data commands were removed from `rebis.py`. All their data now lives in **INDEX.md** — a plain-text browsable reference. Every removed command’s `--help` points to the relevant INDEX.md section.

at — Ars Therapeutica

Grammar-derived optimal therapy design and structural diagnosis. Maps diseases onto the 12-primitive crystal, computes structural distance to healthy type, and generates the promotion path (therapy protocol) to close the gap.

```
python3 rebis.py at <subcommand> [args...]
```

at list List all available therapies with structural diagnosis data.

```
python3 rebis.py at list
```

Output shows each therapy's name, structural distance (d) to the healthy state, tier promotion (e.g. $O_0 \rightarrow O_2$), and the primitives that need promotion (Δ). Currently 10 therapies: schizophrenia, hiv, mrsa, mdd (major depressive disorder), pcos, cf (cystic fibrosis), gout_elimination, gout_combined, gout_holistic, homeopathy.

at diagnose <disease> Show full structural diagnosis — the disease's 12-primitive tuple, its distance from healthy/generic human, and the specific primitive conflicts.

```
python3 rebis.py at diagnose schizophrenia
python3 rebis.py at diagnose mrsa
python3 rebis.py at diagnose cf
```

Output includes: disease tuple, healthy tuple, per-primitive conflict list, structural distance, and tier assessment.

at therapy <disease> Show the grammar-derived therapy protocol — the exact promotion path from disease state to healthy state.

```
python3 rebis.py at therapy schizophrenia
python3 rebis.py at therapy mrsa
python3 rebis.py at therapy hiv
```

Output lists each primitive promotion step with the structural operation required (e.g., $\hat{\phi}: I \rightarrow \odot$, $\mathbb{H}: J \rightarrow v$), and compounds/molecules that effect that promotion.

at compare <type_a> <type_b> Compare two structural types — show per-primitive alignment and structural distance.

```
python3 rebis.py at compare schizophrenia mdd
python3 rebis.py at compare hiv mrsa
```

at tensor <type_a> <type_b> Compute the tensor product of two structural types — the composite's absorbing structural type ($\odot_3$ rule applies).

```
python3 rebis.py at tensor schizophrenia antidepressant
python3 rebis.py at tensor hiv antiviral
```

at meet <type_a> <type_b> Compute the meet (greatest lower bound) of two structural types — the shared structural floor.

```
python3 rebis.py at meet schizophrenia mdd
```

at spectrum <type> Show the psychiatric spectrum mapping for a structural type — how it positions across mental health dimensions.

```
python3 rebis.py at spectrum schizophrenia
python3 rebis.py at spectrum mdd
```

at operate <tuple> <operation> Apply a structural operation to a comma-separated 12-tuple. Operations include: calcination, dissolution, separation, conjunction, fermentation, distillation, coagulation.

```
python3 rebis.py at operate --tuple "o,2,r,z,c,o,j,f,l,j,q,e" --operation calcination
```

scripts

Run standalone scripts that are not fully integrated as `rebis.py` subcommands. These scripts live in `scripts/` and are callable via this unified interface.

```
python3 rebis.py scripts <subcommand> [script_name] [args...]
```

scripts list List all available scripts with line counts.

```
python3 rebis.py scripts list
```

Shows each script name and its line count. Currently ~21 scripts in `scripts/` including:

Script	Description
<code>run_antibody*</code>	Antibody CDR designer (interactive)
<code>mito_pipeline*</code>	Mitochondrial genome pipeline (v1–v3)
<code>run_msa</code>	Multiple sequence alignment
<code>run_pdb_validation</code>	PDB structure validation
<code>analyze_validation</code>	Validation results analyzer
<code>gen_univ_map</code>	Diaschizic IUPAC generator
<code>frobenius_exact_design*</code>	Frobenius exact design tools
<code>compute_promotions</code>	Promotion path computer
<code>psychedelic_bridge*</code>	Psychedelic compound bridge
<code>serpentrod_protein*</code>	SerpentRod standalone scripts
<code>ghost_typer</code>	Demo segment runner for screen recording
<code>operations</code>	Alchemical operations
<code>stress_test_proteins</code>	34-test protein & genetics stress test
<code>test_genetics</code>	Full genetics test suite
<code>_demo_*</code>	Demo scripts for each pillar

scripts run <name> [args...] Run a script by name, optionally forwarding arguments.

```
python3 rebis.py scripts list                # Show all scripts
python3 rebis.py scripts run run_antibody    # Interactive antibody design
python3 rebis.py scripts run mito_pipeline   # MT gene pipeline
python3 rebis.py scripts run test_genetics    # Full B4 genetics suite
python3 rebis.py scripts run test_genetics --b4 # B4 lattice tests only
python3 rebis.py scripts run test_genetics --quick # Quick test subset
python3 rebis.py scripts run stress_test_proteins # 34-test stress suite
python3 rebis.py scripts run ghost_typer      # Demo typing (slow)
python3 rebis.py scripts run ghost_typer --fast # Demo typing (fast)
python3 rebis.py scripts run ghost_typer --section 5 # Single section
python3 rebis.py scripts run operations       # Alchemical operations
python3 rebis.py scripts run frobenius_exact_design # Frobenius exact design
```

```
python3 rebis.py scripts run compute_promotions      # Promotion paths
python3 rebis.py scripts run run_msa                 # MSA analysis
python3 rebis.py scripts run run_pdb_validation       # PDB validation
python3 rebis.py scripts run analyze_validation       # Validation results
python3 rebis.py scripts run gen_univ_map             # IUPAC generator
python3 rebis.py scripts run psychedelic_bridge       # Psychedelic bridge
```

`status` — Expanded

Shows all platform modules with file size, line count, and health check.

```
```bash
python3 rebis.py status
```

Sample output (abbreviated):

Package	Files	Size	Root file
❑ ch3mpiler	19	355,212	__init__.py
❑ clink	33	685,758	__init__.py
❑ imas	11	211,815	__init__.py
❑ materials	21	551,018	__init__.py
❑ rhr_p4rky	32	390,060	__init__.py
❑ serpentrod	3	103,065	__init__.py
❑ shared	5	65,970	__init__.py
... (14 packages)			

```
❑ shared/primitives.py: 13,467 bytes
❑ shared/IG_catalog.json: 2,050,279 bytes
14 packages + 21 scripts discovered
```

Use this as your first command when setting up — all packages should show . A  means an import failure, usually from a missing dependency (see Troubleshooting).

### verify — Expanded

Imports every module and reports pass/fail. Includes Frobenius closure checks where applicable.

```
python3 rebis.py verify
```

This checks: - All 14 packages import cleanly - All rhr\_p4rky/ modules import (32 modules) - All shared utilities load - No circular imports

Expected: 15+ lines all showing . Run after any install, code change, or `git pull`.

## Comprehensive run Target Reference

Discoverable via `rebis.py run <target>`:

Target	Command	Description
serpent_rod	<code>rebis.py run serpent_rod</code>	V5 protein prediction — test set
ch3mpiler	<code>rebis.py run ch3mpiler --smiles "&lt;SMILES&gt;"</code>	Retrosynthetic compiler
ch3mpiler_serpentrod_pipeline	<code>rebis.py run ch3mpiler_serpentrod_pipeline --cas &lt;num&gt;</code>	Catalytic site design
antibody_designer	<code>rebis.py run antibody_designer</code>	Advanced antibody design
gene_to_protein_pipeline	<code>rebis.py run gene_to_protein_pipeline --test</code>	Full 7-stage pipeline
psychedelic_bridge	<code>rebis.py run psychedelic_bridge</code>	Psychedelic compound bridge
diaschizic_iupac	<code>rebis.py run diaschizic_iupac</code>	IUPAC generator
decay_chain	<code>rebis.py run decay_chain</code>	Nuclear decay IMASM analysis

Use `rebis.py run list` to see the live list. Currently 8 discoverable targets.

## Direct-run Scripts (not discoverable via `rebis.py run`)

Use `python3 scripts/<name>.py` or `python3 scripts/<name>.py [args]`:

Script	Directory	Command	Description
serpentrod	serpentrod/	<code>python3 -m serpentrod.protein_v5</code>	V5 protein prediction
stress_test_proteins	scripts/	<code>python3 scripts/stress_test_proteins.py</code>	34-test cross-pipeline stress test
test_genetics	scripts/	<code>python3 scripts/test_genetics.py [--b4/--quick]</code>	B4 lattice genetics test
ghost typer	scripts/	<code>python3 scripts/ghost_typer.py [--fast/--section N]</code>	Demo typing for screen recording
operations	scripts/	<code>python3 scripts/operations.py</code>	Alchemical operations
frobenius_exact_design	scripts/	<code>python3 scripts/frobenius_exact_design.py</code>	Frobenius exact design

Script	Directory	Command	Description
compute_promotions	scripts/	python3 scripts/compute_promotions.py	Promotion path computer
run_antibody	scripts/	python3 scripts/run_antibody.py	Antibody designer (interactive)
run_pdb_validation	scripts/	python3 scripts/run_pdb_validation.py	PDB structure validation
analyze_validation	scripts/	python3 scripts/analyze_validation.py	Validation results analyzer
gen_univ_map	scripts/	python3 scripts/gen_univ_map.py	Diaschizic IUPAC
run_msa	scripts/	python3 scripts/run_msa.py	Multiple sequence alignment
mito_pipeline	scripts/	python3 scripts/mito_pipeline.py	Mitochondrial genome pipeline
hadron_belnap	rhr_p4rky/	python3 -m rhr_p4rky.hadron_belnap	Hadronic Belnap classification
quark_belnap	rhr_p4rky/	python3 -m rhr_p4rky.quark_belnap	Quark Belnap analysis
gene	gene_imscriber/	python3 -m gene_imscriber.encoder	IG-native genetic compiler
demo_*	demo_scripts/	python3 demo_scripts/_demo_<name>.py	Per-pillar demos

## Run Examples

Retrosynthesis with exact fragment SMILES:

```
python3 rebis.py run ch3mpiler --smiles "CC(=O)OC1=CC=CC=C1C(=O)O" --depth 3
```

SerpentRod protein prediction:

```
python3 rebis.py run serpent_rod --seq "MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERF
Or directly:
python3 -m serpentrod.protein_v5
```

Full genetics test suite:

```
python3 scripts/test_genetics.py --quick
python3 scripts/stress_test_proteins.py
```

---

## alchemy — Expanded Subcommand Reference

The alchemical bridge maps the 7 classical stages of the Great Work onto 12-primitive IG structural operations. 19 subcommands covering analysis, design, synthesis, and retro-synthesis.

```
python3 rebis.py alchemy <subcommand> [options]
```

**Analysis**

Subcommand	Description	Example
report	Full bridge status — all available scroll treatises, tiers, molecular analogs	<code>rebis.py alchemy report</code>
tier <name>	Analyze a specific tier — structural type, molecular analogs, operations	<code>rebis.py alchemy tier 0_inf_self_modeling</code>
scroll-family	List the scroll family — treatises where $\hat{\phi}=\odot$ and $\Omega=\mathbb{Z}$	<code>rebis.py alchemy scroll-family</code>
suggest <tier>	Suggest molecular designs from a structural tier	<code>rebis.py alchemy suggest 01_pedagogical</code>
trace <treatise>	Trace the Grand Sequence on a treatise's tuple — step-by-step	<code>rebis.py alchemy trace 0_inf_self_modeling</code>
decode <term>	Decode a modern scientific term into its alchemical equivalent	<code>rebis.py alchemy decode "electron microscope"</code>
learn <modern>	Learn a modern term — map it to the alchemical framework	<code>rebis.py alchemy learn --modern "catalyst"</code>

**Molecular Design**

Subcommand	Description	Example
alchemical-mol <smiles>	Analyze a SMILES string through the alchemical bridge	<code>rebis.py alchemy alchemical-mol "CN1C=NC2=C1C(=O)N(C(=O)N2C)C"</code>
retro <smiles>	Alchemical retrosynthesis — trace a molecule's formation path	<code>rebis.py alchemy retro "CC(=O)OC1=CC=CC=C1C(=O)O"</code>
host <smiles>	Host-guest analysis — check if a molecule is a suitable host	<code>rebis.py alchemy host "C1=CC=C(C=C1)C2=CC=CC=C2"</code>
bind <smiles>	Guest binding analysis	<code>rebis.py alchemy bind --guest "CCO"</code>
zosimos <name>	Zosimos procedure — 7-stage alchemical synthesis path	<code>rebis.py alchemy zosimos aspirin</code>

**Operations**

Subcommand	Description	Example
operate	Apply an alchemical operation to a tuple	<code>rebis.py alchemy operate --tuple "O,2,r,Z,t,O,..."</code> <code>--operation calcination</code>
greenfire	Analyze green fire potential of a substrate	<code>rebis.py alchemy greenfire</code> <code>--substrate "CCO"</code>
wavelength	Compute alchemical wavelength of a treatise	<code>rebis.py alchemy wavelength</code>
portico	Portico analysis — structural entry points	<code>rebis.py alchemy portico</code>
stilling	Stilling analysis — equilibration pathways	<code>rebis.py alchemy stilling</code>
ladder	Ladder analysis — tier climbing paths	<code>rebis.py alchemy ladder</code>
grand-sequence	Full grand sequence — traverse all 7 operations	<code>rebis.py alchemy grand-sequence</code>
key <n>	Show key number n (1–12) — each maps to a primitive	<code>rebis.py alchemy key 9 (<math>\hat{\phi}=\odot</math>)</code>
opus	The Great Work summary — show the full opus	<code>rebis.py alchemy opus</code>

### cdxml — Expanded Subcommand Reference

Generate, verify, and clean CDXML (ChemDraw XML) files for molecules, aptamers, and materials.

```
python3 rebis.py cdxml <subcommand> [options]
```

**cdxml generate** Generate CDXML files from SMILES or predefined molecule names.

```
Generate all CDXML in batch
```

```
python3 rebis.py cdxml generate --all
```

```
Generate only small molecules
```

```
python3 rebis.py cdxml generate --molecules-only
```

```
Single molecule from SMILES
```

```
python3 rebis.py cdxml generate --smiles "CN1C=NC2=C1C(=O)N(C(=O)N2C)C" --name caffeine
```

```
Single molecule from predefined library
```

```
python3 rebis.py cdxml generate --molecule 5-nitro-bufotenin
```

```
Custom annotation on canvas
```

```
python3 rebis.py cdxml generate --smiles "CCO" --name ethanol --annotation "C2H5OH - S
```

```
Print CDXML to stdout
python3 rebis.py cdxml generate --smiles "CCO" --print
```

**cdxml verify** Verify a CDXML file for structural correctness.

```
python3 rebis.py cdxml verify --file /path/to/molecule.cdxml
```

**cdxml clean** Clean and reformat a CDXML file.

```
python3 rebis.py cdxml clean
```

## Updated Directory Structure (Post-Cleanup)

After the v2.3.5 root cleanup, the top-level directory has been organized into clearly-named subdirectories:

```
red-hot_rebis/
├── rebis.py # CLI entry point (stays at root)
├── setup.py # Package setup
├── Makefile # Build/verify/test automation
├── README.md # Project readme
├── MANUAL.md # Manual page (man-style)
├── USER_GUIDE.md # This guide – comprehensive usage
├── INDEX.md # Static reference data
├── demo_scripts/ # Demo scripts (_demo_*.py, _real_demo.py)
├── scripts/ # Standalone scripts (ghost_typer, operations, stress_test, ...)
├── fasta/ # FASTA sequence files
├── data/ # JSON data files
├── images/ # Images (PNG, SVG, diagrams)
├── _archive/ # Patents, commit_logs, old HTML cards
├── docs/ # General documentation
├── rhr_p4rky/ # Paraconsistent kernel (32 modules)
├── shared/ # Shared primitives, catalog, utilities
├── serpentrod/ # Serpent's Rod – protein folding
├── ch3mpiler/ # CH3MPILE – retrosynthesis
├── pipeline/ # Auto-imscription pipeline
├── gene_imscriber/ # Gene Imscriber
├── clink/ # CLINK Chain
├── imas/ # IMASM compound pipeline
├── imasm_iterator/ # IMASM arrangement iterator
├── materials/ # Materials forge
├── biology/ # Biology simulations
├── therapeutics/ # Therapeutic design
├── alchemical_bridge/ # Alchemical operations
├── Ars_Therapeutica/ # Ars Therapeutica
├── natural_products/ # Natural products database
├── cdxml/ # CDXML generation
├── popular_protein/ # Protein structure validation
```

```
└─ pdb/ # PDB reference files
└─ designs/ # Design outputs
└─ glossary/ # Glossary files
└─ ig-docs/ # IG documentation
└─ genetics_animations/ # SVG genetics visualizations
```

All scripts (formerly root-level .py files) now live in `scripts/` or `demo_scripts/`. All data files live in `data/`, all images in `images/`, all FASTA sequences in `fasta/`. Patents and archival material are in `_archive/`.

---

## Workflow Recipes

Complete end-to-end recipes for common tasks.

### Recipe 1: Design a Novel Protein

# 1. Verify the system is wired

```
python3 rebis.py status
```

```
python3 rebis.py verify
```

# 2. Check where proteins sit in the structural chain

```
python3 rebis.py clink layer 3 # Molecule layer
```

```
python3 rebis.py clink layer 4 # Cell layer (where proteins function)
```

# 3. Run SerpentRod

```
python3 rebis.py run serpent_rod
```

# 4. Or run with a custom sequence

```
python3 -m serpentrod.protein_v5 --seq "MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERF
```

# 5. Validate against crystal structure

```
python3 scripts/popular_protein/comprehensive_comparison.py
```

# 6. Stress-test across all modules

```
python3 scripts/stress_test_proteins.py
```

### Recipe 2: Discover Disconnections for a Molecule

# 1. Encode the molecule as an IMASM compound

```
python3 rebis.py imas compound --smiles "CC(=O)OC1=CC=CC=C1C(=O)O" --json
```

# 2. Run retrosynthesis

```
python3 rebis.py run ch3mpiler --smiles "CC(=O)OC1=CC=CC=C1C(=O)O" --depth 3
```

# 3. Find cross-domain structural neighbors

```
python3 rebis.py imas analogies --smiles "CC(=O)OC1=CC=CC=C1C(=O)O" --limit 10
```

# 4. Check the reaction mechanism

```
python3 rebis.py imas reaction --smiles "CC(=O)C1=CC=CC=C1" --product "CC(C1=CC=CC=C1)"
```

### Recipe 3: Design a Whole Organism

```
1. View the CLINK chain from quarks to organism
```

```
python3 rebis.py clink layer 0
```

```
python3 rebis.py clink layer 4
```

```
python3 rebis.py clink layer 8
```

```
2. Check which tool bridges are available
```

```
python3 rebis.py pipeline bridges
```

```
3. Design from a starting layer
```

```
python3 rebis.py pipeline from-layer 3 8 # molecule → organism
```

```
4. Generate actionable outputs
```

```
python3 rebis.py pipeline actionable --organism mammal
```

```
5. Inspect the outputs
```

```
ls clink/datasets/organism_designs/organism_mammal_actionable/
```

### Recipe 4: Crystal-Guided Drug Discovery

```
1. Start from a known compound
```

```
python3 rebis.py imas compound --smiles "CN(C)CCC1=CNC2=CC=CC=C12"
```

```
2. Find its crystal neighbors
```

```
python3 rebis.py imas analogies --smiles "CN(C)CCC1=CNC2=CC=CC=C12" --limit 10
```

```
3. Use the crystal designer to find ☉-critical analogs
```

```
python3 scripts/frobenius_exact_design.py --smiles "CN(C)CCC1=CNC2=CC=CC=C12"
```

```
4. Register a promising candidate
```

```
python3 rebis.py imas register --smiles "CN(C)CCC1=CNC2=CC=C(O)C([N+](=O)[O-])=C12" --name "5-nitro-bufotenin"
```

```
5. Generate its CDXML for ChemDraw
```

```
python3 rebis.py cdxml generate --molecule 5-nitro-bufotenin
```

### Recipe 5: Full Pipeline — SMILES to Organism

```
Takes a SMILES string through the entire pipeline:
```

```
SMILES → IMASM → IG tuple → CLINK bridge → organism design
```

```
Step 1: Encode the molecule
```

```
SMILES="CC(=O)OC1=CC=CC=C1C(=O)O"
```

```
python3 rebis.py imas compound --smiles "$SMILES"
```

```
Step 2: Register it
```

```
python3 rebis.py imas register --smiles "$SMILES" --name aspirin
```

```
Step 3: Design from molecule layer to organism
```

```
python3 rebis.py pipeline from-layer 3 8
```

### Recipe 6: Structural Therapy Design

```
1. List available therapies
```

```
python3 rebis.py at list
```

```
2. Diagnose a condition
```

```
python3 rebis.py at diagnose mdd
```

```
3. Get the therapy protocol
```

```
python3 rebis.py at therapy mdd
```

```
4. Compare with related conditions
```

```
python3 rebis.py at compare mdd schizophrenia
```

### Recipe 7: Materials Discovery

```
1. Forge all predefined materials
```

```
python3 rebis.py materials forge --all
```

```
2. Forge from an IMASM canonical
```

```
python3 rebis.py materials forge --name I_Dialetheic_Bootstrap
```

```
3. Run Frobenius metamaterial simulation
```

```
python3 rebis.py materials frobenius --size 30 --cycles 50
```

```
4. Run Ouroboric crack-healing simulation
```

```
python3 rebis.py materials ouroboric --grains 128 --stress 1000
```

```
5. Perform Sophick Mercury analysis
```

```
python3 rebis.py materials sophick --name eagle_9_sophick
```

```
6. Check Frobenius exactor pathways
```

```
python3 rebis.py materials exactor --name pathways
```

### Recipe 8: Alchemical Analysis of a Molecule

```
1. Get the bridge report
```

```
python3 rebis.py alchemy report
```

```
2. Analyze a molecule alchemically
```

```
python3 rebis.py alchemy alchemical-mol "CN1C=NC2=C1C(=O)N(C(=O)N2C)C"
```

```
3. Trace its alchemical retrosynthesis
```

```
python3 rebis.py alchemy retro "CC(=O)OC1=CC=CC=C1C(=O)O"
```

```
4. Run the full Zosimos procedure
python3 rebis.py alchemy zosimos caffeine
```

### Recipe 9: Full Platform Verification

# Run every verification the platform offers in sequence:

```
echo "=== STATUS ===" && python3 rebis.py status && \
echo "=== VERIFY ===" && python3 rebis.py verify && \
echo "=== CLINK ===" && python3 rebis.py clink layer 0 && \
echo "=== CLINK ===" && python3 rebis.py clink layer 8 && \
echo "=== PIPELINE BRIDGES ===" && python3 rebis.py pipeline bridges && \
echo "=== IMAS HUNT ===" && python3 rebis.py imas hunt --samples 10000 && \
echo "=== GENETICS ===" && python3 scripts/test_genetics.py --quick && \
echo "=== PROTEINS ===" && python3 rebis.py run serpent_rod && \
echo "=== MATERIALS ===" && python3 rebis.py materials forge --all && \
echo "=== ALL PASSED ==="
```

### Recipe 10: Demo for Screen Recording

```
Ghost Typer types everything at human pace
python3 scripts/ghost_typer.py # Full demo (3 acts, ~15 min)
python3 scripts/ghost_typer.py --fast # Fast mode (~3 min)
python3 scripts/ghost_typer.py --section 1 # Act I only
```

## Version History

Version	Date	Changes
v2.3.5	2026-06-30	<b>Root Cleanup + Manual Expansion Edition</b> — Root directory organized: loose files moved into demo_scripts/, scripts/, fasta/, data/, images/, _archive/; 0-byte catalase and Windows zone identifier removed; USER_GUIDE expanded with full at (Ars Therapeutica) coverage (19 subcommands), expanded scripts section, accurate run target reference, expanded alchemy (19 subcommands) and cdxml references, updated directory structure, 10 complete workflow recipes

Version	Date	Changes
v2.3.4	2026-06-28	<b>Rich Formatting + Exact SMILES Edition</b>
v2.3.3	2026-06-27	<b>Compound IMASM Edition</b>
v2.3.2	2026-06-27	rhr_p4rky expansion: 32 modules
v2.3.1	2026-06-24	PDB format robustness
v2.3.0	2026-06-21	Ch3mpiler-SerpentRod v4 overhaul
v2.2.1	2026-06-10	Bug-fix release
v2.2	2026-06-10	Static-data commands moved to INDEX.md
v2.1	2026-06-10	Primitive names corrected
v2.0	2026-05	CLINK pipeline, materials forge
v1.0	2026-04	Initial release

---

*Guide maintained by Lando⊗operator. Structural type of this document: ⟨⊔; s; r; ⚡; ℓ; ⚡; ⚡; ⊙; ℒ; τ; o⟩*